

PYTHON PRACTICE

Exercise 1 - List Basics

Create a list of 8 city names.

Write code to:

1. Print the first and last city using indexing.
2. Print cities from index 2 to 5 using slicing.
3. Add a new city at the end.
4. Insert a city at index 2.
5. Remove a city by name using remove().
6. Remove and print the last city using pop().
7. Print the total count of cities.
8. Check if "Pune" is in the list. Print True or False.

Concepts: indexing, slicing, append(), insert(),

remove(), pop(), len(), in

```
# =====
# Exercise 1 - List Basics

city_name = ['Pune', 'Chh.sambhajinagar', 'Jalna', 'Mumbai', 'Kolhapur', 'Kerala', 'ujjain', 'kedarnath']

# 1. Print the first and last city using indexing.
print(f"First city is: \'{city_name[0]}\' and Last city is: \'{city_name[-1]}\'")

print("====="*6)
# 2. Print cities from index 2 to 5 using slicing.
print(f"Cities from index 2 to 5 is: {city_name[2:5]}")

print("====="*6)
# 3. Add a new city at the end.
city_name.append('Goa')
print(f"After append method list is: {city_name}")

print("====="*6)
# 4. Insert a city at index 2.
city_name.insert(2, 'Kolkata')
print(f"After insert method list is: {city_name}")

print("====="*6)
# 5. Remove a city by name using remove().
city_name.remove('Kolkata')
print(f"After remove method list is: {city_name}")
```

```

print("====="*6)
# 6. Remove and print the last city using pop().
popped_items = city_name.pop(-1)
print(f"Popped element is: {popped_items}")
print(f"After pop method list is: {city_name}")

print("====="*6)
# 7. Print the total count of cities.
count = 0
for i in city_name:
    count += 1
print(f"Total city count is: {count}")

print(f"Total city count is: {len(city_name)}")

print("====="*6)
# 8. Check if "Pune" is in the list. Print True or False.
if 'Pune' in city_name:
    print("True")
else:
    print("False")

```

Output :-

```

First city is: 'Pune' and Last city is: 'kedarnath'
=====
Cities from index 2 to 5 is: ['Jalna', 'Mumbai', 'Kolhapur']
=====
After append method list is: ['Pune', 'Chh.sambhajinagar', 'Jalna', 'Mumbai', 'Kolhapur', 'Kerala', 'ujjain', 'k
=====
After insert method list is: ['Pune', 'Chh.sambhajinagar', 'Kolkata', 'Jalna', 'Mumbai', 'Kolhapur', 'Kerala', '
=====
After remove method list is: ['Pune', 'Chh.sambhajinagar', 'Jalna', 'Mumbai', 'Kolhapur', 'Kerala', 'ujjain', 'k
=====
Popped element is: Goa
After pop method list is: ['Pune', 'Chh.sambhajinagar', 'Jalna', 'Mumbai', 'Kolhapur', 'Kerala', 'ujjain', 'keda
=====
Total city count is: 8
Total city count is: 8
=====
True

```

Exercise 2 - List Operations

Given this list of sales figures:

[1500, 4200, 3100, 870, 5600, 2200, 4800, 990]

Without using built-in sort():

1. Find the maximum value using a loop.
2. Find the minimum value using a loop.
3. Calculate the total and average.
4. Count how many values are above 3000.
5. Create a new list containing only values above 3000.

Print all results with clear labels.

Concepts: loop, condition, counter, new list, arithmetic

1. Find the maximum value using a loop.

```
...  
  
# Exercise 2 - List Operations  
sales_figures = [1500, 4200, 3100, 870, 5600, 2200, 4800, 990]  
# 1. Find the maximum value using a loop.  
max_no = 0  
for i in sales_figures:  
    if i >= max_no:  
        max_no = i  
print(f"Maximum Number is: {max_no}")
```

✓ 0.0s

Maximum Number is: 5600

2. Find the minimum value using a loop.

```
# 2. Find the minimum value using a loop.  
min_no = sales_figures[0]  
for i in sales_figures:  
    if i < min_no:  
        min_no = i  
print(f"Minimum Number is: {min_no}")
```

✓ 0.0s

Minimum Number is: 870

3. Calculate the total and average.

```
# 3. Calculate the total and average.
sum = 0
total = 0
for i in sales_figures:
    sum += i
    total += 1
print(f"Total is {sum}, Average of number is: {sum/total}")
```

✓ 0.0s

Total is 23260, Average of number is: 2907.5

4. Count how many values are above 3000.

```
# 4. Count how many values are above 3000.
count = 0
for i in sales_figures:
    if i > 3000:
        count += 1
    else:
        continue
print(f"Above 3000 values count is: {count}")
```

✓ 0.0s

Above 3000 values count is: 4

5. Create a new list containing only values above 3000.

```
# 5. Create a new list containing only values above 3000.
count = 0
new_list = []
for i in sales_figures:
    if i > 3000:
        new_list.append(i)
    else:
        continue
print(f"Above 3000 values list: {new_list}")
```

✓ 0.0s

Above 3000 values list: [4200, 3100, 5600, 4800]

Exercise 3 - List of Lists

You have a dataset as a list of lists:

```
students = [  
    ["Rahul", "IT", 85],  
    ["Amit", "HR", 62],  
    ["Priya", "IT", 91],  
    ["Neha", "Finance", 74],  
    ["Karan", "IT", 58],  
    ["Sneha", "HR", 88]  
]
```

Write code to:

1. Print all student names.
2. Print students who scored above 75.
3. Print all students from the IT department.
4. Find and print the student with the highest score.
5. Count students per department.

Output format:

IT : 3

HR : 2

Finance : 1

Concepts: nested list, indexing, loop, condition, dict

1. Print all student names.

```
students = [  
    ["Rahul", "IT", 85],  
    ["Amit", "HR", 62],  
    ["Priya", "IT", 91],  
    ["Neha", "Finance", 74],  
    ["Karan", "IT", 58],  
    ["Sneha", "HR", 88]  
]  
  
# 1. Print all student names.  
column = [row[0] for row in students]  
print(f"All student names: {column}")
```

✓ 0.0s

All student names: ['Rahul', 'Amit', 'Priya', 'Neha', 'Karan', 'Sneha']

2. Print students who scored above 75.

```
# 2. Print students who scored above 75.  
for i in students:  
    if int(i[2]) > 75:  
        print(i)
```

✓ 0.0s

```
['Rahul', 'IT', 85]  
['Priya', 'IT', 91]  
['Sneha', 'HR', 88]
```

3. Print all students from the IT department.

```
# 3. Print all students from the IT department.  
for column in students:  
    if column[1] == 'IT':  
        print(column)
```

✓ 0.0s

```
['Rahul', 'IT', 85]  
['Priya', 'IT', 91]  
['Karan', 'IT', 58]
```

4. Find and print the student with the highest score.

```
# 4. Find and print the student with the highest score.  
max_score = students[0]  
for i in students:  
    if i[2] > max_score[2]:  
        max_score = i  
    else:  
        continue  
print(f"Highest score student is: {max_score}")
```

✓ 0.0s

Highest score student is: ['Priya', 'IT', 91]

5. Count students per department.

```
# 5. Count students per department.  
count_dep = {}  
for i in students:  
    if i[1] in count_dep:  
        count_dep[i[1]] += 1  
    else:  
        count_dep[i[1]] = 1  
print(count_dep)
```

✓ 0.0s

{'IT': 3, 'HR': 2, 'Finance': 1}

Exercise 4 - String Methods Practice

Given this raw string:

```
raw = " rahul sharma , data engineer , pune , 85000 "
```

Write code to:

1. Remove leading and trailing spaces using strip().
2. Split the string by comma into a list.
3. Strip spaces from each item in the list.
4. Convert name to Title Case.
5. Convert designation to uppercase.
6. Convert city to lowercase.
7. Check if the salary field contains only digits
after stripping. Use isdigit().
8. Rejoin the cleaned values with " | " as separator.

Expected final output:

Rahul Sharma | DATA ENGINEER | pune | 85000

```

raw = " rahul sharma , data engineer , pune , 85000 "
# 1. Remove leading and trailing spaces using strip().
new_raw = raw.strip()
# 2. Split the string by comma into a list.
clean = raw.split(',')
# 3. Strip spaces from each item in the list.
data = [i.strip() for i in clean]
# 4. Convert name to Title Case.
data[0] = data[0].title()
# 5. Convert designation to uppercase.
data[1] = data[1].upper()
# 6. Convert city to lowercase.
data[2] = data[2].lower()
# 7. Check if the salary field contains only digits
#     after stripping. Use isdigit().
if data[3].isdigit():
    salary = data[3]
else:
    salary = "Invalid salary"
# 8. Rejoin the cleaned values with " | " as separator.
new_string = ' | '.join(data)
print(new_string)

```

✓ 0.0s

Rahul Sharma | DATA ENGINEER | pune | 85000

Exercise 5 - String Search and Replace

Given this sentence:

text = "Data engineering involves data ingestion,
data transformation, and data storage."

Write code to:

1. Count how many times the word "data" appears.
Case-insensitive count.
2. Replace every occurrence of "data" with "DATA".
3. Check if the sentence starts with "Data".
4. Check if the sentence ends with "storage.".
5. Find the index position of the first occurrence
of "transformation".
6. Extract the word "transformation" using slicing based on the index you found.

#1. Count how many times the word "data" appears. Case-insensitive count.

```
text = "Data engineering involves data ingestion,data transformation, and data storage."

#1. Count how many times the word "data" appears. Case-insensitive count.
count = text.lower().count("data")
print(f"Count of data words: {count}")
```

✓ 0.0s

Count of data words: 4

2. Replace every occurrence of "data" with "DATA".

```
text = "Data engineering involves data ingestion,data transformation, and data storage."

# 2. Replace every occurrence of "data" with "DATA".
new_data = text.replace('data', 'Data')
print(f"After replace mwthod is: {new_data}")
```

✓ 0.0s

After replace mwthod is: Data engineering involves Data ingestion,Data transformation, and Data storage.

3. Check if the sentence starts with "Data".

```
text = "Data engineering involves data ingestion,data transformation, and data storage."

# 3. Check if the sentence starts with "Data".
print(f"Check sentence starts with Data: {text.startswith("Data")}")
```

✓ 0.0s

Check sentence starts with Data: True

4. Check if the sentence ends with "storage."

```
text = "Data engineering involves data ingestion,data transformation, and data storage."

# 4. Check if the sentence ends with "storage."
print(f"Check sentence ends with storage: {text.endswith("storage.")}")
```

✓ 0.0s

Check sentence ends with storage: True

5. Find the index position of the first occurrence of "transformation".

```
text = "Data engineering involves data ingestion,data transformation, and data storage."

# 5. Find the index position of the first occurrence of "transformation".
index = text.find("transformation")
print(f"Index is: {index}") |
```

✓ 0.0s

Index is: 46

6. Extract the word "transformation" using slicing based on the index you found.

```
text = "Data engineering involves data ingestion,data transformation, and data storage."

# 6. Extract the word "transformation" using slicing based on the index you found.
extract_word = text[index:index+len('transformation')]
print(extract_word)
```

✓ 0.0s

transformation

Exercise 6 - Dictionary Basics

Create a dictionary to store details of one employee:

name, department, salary, city, is_active

Write code to:

1. Print all keys.
2. Print all values.
3. Print all key-value pairs using items().
4. Update the salary to a new value.
5. Add a new key: years_of_experience with value 3.
6. Delete the is_active key.
7. Check if the key "department" exists.
8. Get the value of a key that does not exist.

Use .get() with a default value of "Not Found".

1. Print all keys.

```
employee = {  
    "name": "Rahul",  
    "department": "IT",  
    "salary": 75000,  
    "city": "Jalna",  
    "is_active": 1 # 1 is 'True' or 0 is 'False'  
}  
  
# 1. Print all keys.  
print(employee.keys())
```

✓ 0.0s

```
dict_keys(['name', 'department', 'salary', 'city', 'is_active'])
```

2. Print all values.

```
# 2. Print all values.  
print(employee.values())
```

✓ 0.0s

```
dict_values(['Rahul', 'IT', 75000, 'Jalna', 1])
```

3. Print all key-value pairs using items().

```
# 3. Print all key-value pairs using items().  
print(employee.items())
```

✓ 0.0s

Python

```
dict_items([('name', 'Rahul'), ('department', 'IT'), ('salary', 75000), ('city', 'Jalna'), ('is_active', 1)])
```

4. Update the salary to a new value.

```
# 4. Update the salary to a new value.  
employee.update({'salary': 89000})  
print(employee)
```

✓ 0.0s

```
{'name': 'Rahul', 'department': 'IT', 'salary': 89000, 'city': 'Jalna', 'is_active': 1}
```

5. Add a new key: years_of_experience with value 3.

```
# 5. Add a new key: years_of_experience with value 3.
employee['years_of_experience'] = 3
print(employee)
```

✓ 0.0s Python

```
{'name': 'Rahul', 'department': 'IT', 'salary': 75000, 'city': 'Jalna', 'is_active': 1, 'years_of_experience': 3}
```

6. Delete the is_active key.

```
# 6. Delete the is_active key.

# employee.__delitem__('is_active')
# print(employee)

del employee['is_active']
print(employee)
```

✓ 0.0s

```
{'name': 'Rahul', 'department': 'IT', 'salary': 75000, 'city': 'Jalna'}
```

7. Check if the key "department" exists.

```
# 7. Check if the key "department" exists.
"department" in employee

# if "department" in employee:
#     print("Key is exists")
# else:
#     print("Key is not exists!")
```

✓ 0.0s

True

8. Get the value of a key that does not exist. Use .get() with a default value of "Not Found".

```
# 8. Get the value of a key that does not exist. Use .get() with a default value of "Not Found".
employee_get = employee.get('name')
print(employee_get)

employee_get_default = employee.get("favorite_color", "Not Found!")
print(employee_get_default)
```

✓ 0.0s

Rahul
Not Found!

Exercise 7 - Dictionary from Two Lists

You have two lists:

```
columns = ["emp_id", "name", "department", "salary", "city"]
```

```
values = [101, "Priya Mehta", "IT", 85000, "Pune"]
```

Write code to:

1. Create a dictionary by combining both lists.

Do not hardcode. Use zip().

2. Print the dictionary.

3. Print only the name and salary.

4. Convert the salary from integer to a formatted

string: "Rs. 85,000"

1. Create a dictionary by combining both lists. Do not hardcode. Use zip().

```
columns = ["emp_id", "name", "department", "salary", "city"]
values = [101, "Priya Mehta", "IT", 85000, "Pune"]

# 1. Create a dictionary by combining both lists. Do not hardcode. Use zip().
for key, value in zip(columns, values):
    print(key, ': ', value)

dictionary = {}
dictionary = dict(zip(columns, values))
print(dictionary)
```

✓ 0.0s

```
emp_id : 101
name : Priya Mehta
department : IT
salary : 85000
city : Pune
{'emp_id': 101, 'name': 'Priya Mehta', 'department': 'IT', 'salary': 85000, 'city': 'Pune'}
```

3. Print only the name and salary.

```
# 3. Print only the name and salary.
print(f"Name : {dictionary['name']} \nSalary: {dictionary['salary']}")
```

✓ 0.0s

```
Name : Priya Mehta
Salary: 85000
```

4. Convert the salary from integer to a formatted

string: "Rs. 85,000"

```
# 4. Convert the salary from integer to a formatted
# string: "Rs. 85,000"

print(f"\Rs. {dictionary['salary']}\")
```

✓ 0.0s

'Rs. 85000'

Exercise 8 - Nested Dictionary

Create a nested dictionary for 3 students.

Each student should have: marks, city, department.

```
students = {
    "Rahul": {"marks": 85, "city": "Pune",
              "department": "IT"},
    "Amit" : {"marks": 62, "city": "Mumbai",
              "department": "HR"},
    "Priya": {"marks": 91, "city": "Pune",
              "department": "IT"}
}
```

Write code to:

1. Print Rahul's city.
2. Print all student names and their marks.
3. Find the student with the highest marks.
4. Print all students from Pune.
5. Add a new student: Neha, marks 74,
city Delhi, department Finance.

```

students = {
    "Rahul": {"marks": 85, "city": "Pune",
              "department": "IT"},
    "Amit" : {"marks": 62, "city": "Mumbai",
              "department": "HR"},
    "Priya": {"marks": 91, "city": "Pune",
              "department": "IT"}
}
# 1. Print Rahul's city.
print(f"Rahul city is: {students['Rahul']['city']}")
# 2. Print all student names and their marks.
for i in students:
    print(f"Name: {i} Marks: {students[i]['marks']}")
# 3. Find the student with the highest marks.
highest_marks = max(students, key = lambda x: students[x]['marks'])
print(f"Highest marks student name is: {highest_marks}")

# 4. Print all students from Pune.
for i in students:
    if students[i]['city'] == 'Pune':
        print(f"Name: {i} City is: {students[i]['city']}")

```

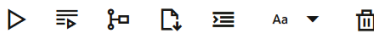
✓ 0.0s

```

Rahul city is: Pune
Name: Rahul Marks: 85
Name: Amit Marks: 62
Name: Priya Marks: 91
Highest marks student name is: Priya
Name: Rahul City is: Pune
Name: Priya City is: Pune

```

SQL PRACTICE

[SQL Worksheet]*   

```
1 CREATE TABLE EMPLOYEES06(emp_id NUMBER, name VARCHAR2(20), department VARCHAR2(20), salary NUMBER, city VARCHAR2(20), join_year NUMBER);
2 INSERT INTO EMPLOYEES06 VALUES(1, 'Rahul', 'IT', 72000, 'Pune', 2021);
3 INSERT INTO EMPLOYEES06 VALUES(2, 'Amit', 'HR', 48000, 'Mumbai', 2020);
4 INSERT INTO EMPLOYEES06 VALUES(3, 'Priya', 'IT', 85000, 'Pune', 2022);
5 INSERT INTO EMPLOYEES06 VALUES(4, 'Neha', 'Finance', 61000, 'Delhi', 2021);
6 INSERT INTO EMPLOYEES06 VALUES(5, 'Karan', 'IT', 55000, 'Bangalore', 2023);
7 INSERT INTO EMPLOYEES06 VALUES(6, 'Sneha', 'HR', 52000, 'Mumbai', 2022);
8 INSERT INTO EMPLOYEES06 VALUES(7, 'Rohan', 'Finance', 47000, 'Pune', 2020);
9 INSERT INTO EMPLOYEES06 VALUES(8, 'Divya', 'IT', 91000, 'Pune', 2023);
10 SELECT * FROM EMPLOYEES06;
```

Query result Script output DBMS output Explain Plan SQL history

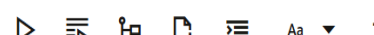
  Download Execution time: 0.01 seconds

	EMP_ID	NAME	DEPARTMENT	SALARY	CITY	JOIN_YEAR
1	1	Rahul	IT	72000	Pune	2021
2	2	Amit	HR	48000	Mumbai	2020
3	3	Priya	IT	85000	Pune	2022
4	4	Neha	Finance	61000	Delhi	2021

SQL Exercise 1 - Basic GROUP BY

Write queries for the following:

1. Count the number of employees in each department.
2. Find the total salary paid per department.
3. Find the average salary per department.
4. Find the minimum and maximum salary per department.
5. Count the number of employees in each city.

[SQL Worksheet]*  

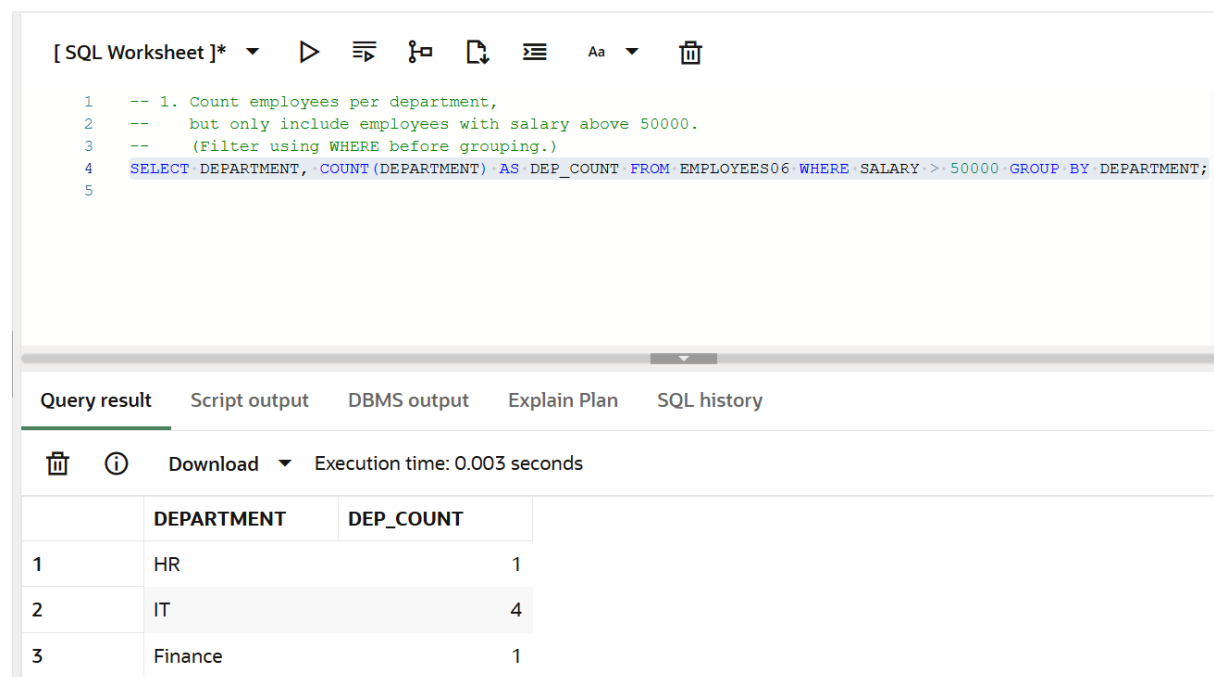
```
1 -- SQL Exercise 1 - Basic GROUP BY
2
3 -- 1. Count the number of employees in each department.
4 SELECT DEPARTMENT, COUNT(DEPARTMENT) AS COUNT FROM EMPLOYEES06 GROUP BY DEPARTMENT;
5
6 -- 2. Find the total salary paid per department.
7 SELECT DEPARTMENT, SUM(SALARY) AS TOTAL_SALARY FROM EMPLOYEES06 GROUP BY DEPARTMENT;
8
9 -- 3. Find the average salary per department.
10 SELECT DEPARTMENT, AVG(SALARY) AS TOTAL_SALARY FROM EMPLOYEES06 GROUP BY DEPARTMENT;
11
12 -- 4. Find the minimum and maximum salary per department.
13 SELECT DEPARTMENT, MIN(SALARY) AS MINIMUM_SALARY, MAX(SALARY) AS MAXIMUM_SALARY FROM EMPLOYEES06 GROUP BY DEPARTMENT;
14
15 -- 5. Count the number of employees in each city.
16 SELECT DEPARTMENT, COUNT('CITY') AS CITY_COUNT FROM EMPLOYEES06 GROUP BY DEPARTMENT;
```


SQL Exercise 2 - GROUP BY with WHERE

Write queries for the following:

1. Count employees per department,
but only include employees with salary above 50000.
(Filter using WHERE before grouping.)
2. Find average salary per city,
only for employees who joined after 2020.
3. Find total salary per department
only for IT and Finance departments.
Use WHERE with IN.

-- 1. Count employees per department,
-- but only include employees with salary above 50000.
-- (Filter using WHERE before grouping.)



The screenshot shows a SQL IDE interface. At the top, there's a toolbar with icons for running, saving, and other functions. Below the toolbar, the SQL query is written in a text area. The query is: `SELECT DEPARTMENT, COUNT(DEPARTMENT) AS DEP_COUNT FROM EMPLOYEES06 WHERE SALARY > 50000 GROUP BY DEPARTMENT;`. Below the query, there's a tabbed interface with 'Query result' selected. Under 'Query result', there's a table with 3 columns: 'DEPARTMENT', 'DEP_COUNT', and an index column. The table contains 3 rows of data: HR (1), IT (4), and Finance (1). The execution time is 0.003 seconds.

```
1  -- 1. Count employees per department,
2  -- but only include employees with salary above 50000.
3  -- (Filter using WHERE before grouping.)
4  SELECT DEPARTMENT, COUNT(DEPARTMENT) AS DEP_COUNT FROM EMPLOYEES06 WHERE SALARY > 50000 GROUP BY DEPARTMENT;
5
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.003 seconds

	DEPARTMENT	DEP_COUNT
1	HR	1
2	IT	4
3	Finance	1

- 2. Find average salary per city,
- only for employees who joined after 2020.

```

4  SELECT DEPARTMENT, COUNT(DEPARTMENT) AS DEP_COUNT FROM EMPLOYEES06 WHERE SALARY > 50000 GROUP BY DEPARTMENT;
5  -- 2. Find average salary per city,
6  -- only for employees who joined after 2020.
7  SELECT CITY, AVG(SALARY) FROM EMPLOYEES06 WHERE JOIN_YEAR > 2020 GROUP BY CITY;
8  SELECT * FROM EMPLOYEES06;

```

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.001 seconds

	CITY	AVG(SALARY)	
1	Bangalore	55000	
2	Mumbai	52000	
3	Delhi	61000	
4	Pune	82666.66666666667	

- 3. Find total salary per department
- only for IT and Finance departments.
- Use WHERE with IN.

```

9  -- 3. Find total salary per department
10 -- only for IT and Finance departments.
11 -- Use WHERE with IN.
12 SELECT DEPARTMENT, SUM(SALARY) FROM EMPLOYEES06
13 WHERE DEPARTMENT IN('IT','Finance')
14 GROUP BY DEPARTMENT;
15

```

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.002 seconds

	DEPARTMENT	SUM(SALARY)
1	Finance	108000

SQL Exercise 3 - HAVING

Write queries for the following:

1. Show only departments that have more than 2 employees.
2. Show departments where the average salary is above 60000.
3. Show cities where the total salary bill exceeds 150000.
4. Show departments where the minimum salary is below 50000.

-- 1. Show only departments that have more than
-- 2 employees.

[SQL Worksheet]*      Aa 

```
1  -- 1. Show only departments that have more than
2  -- 2 employees.
3
4  SELECT DEPARTMENT, COUNT(*) AS TOTAL_EMP
5  FROM EMPLOYEES06
6  GROUP BY DEPARTMENT
7  HAVING COUNT(*) > 2;
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.003 seconds

	DEPARTMENT	TOTAL_EMP
1	IT	4

-- 2. Show departments where the average salary
-- is above 60000.

[SQL Worksheet]*      Aa 

```
1  -- 2. Show departments where the average salary
2  -- is above 60000.
3  SELECT DEPARTMENT FROM EMPLOYEES06
4  GROUP BY DEPARTMENT
5  HAVING AVG(SALARY) > 60000;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

  Download  Execution time: 0.003 seconds

	DEPARTMENT
1	IT

-- 3. Show cities where the total salary bill
-- exceeds 150000.

[SQL Worksheet]*      Aa 

```
1  -- 3. Show cities where the total salary bill
2  -- exceeds 150000.
3  SELECT CITY, SUM(SALARY) AS TOTAL_SALARY FROM EMPLOYEES06
4  GROUP BY CITY
5  HAVING SUM(SALARY) > 150000;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

  Download  Execution time: 0.002 seconds

	CITY	TOTAL_SALARY
1	Pune	295000

-- 4. Show departments where the minimum salary

-- is below 50000.

```
1  -- 4. Show departments where the minimum salary
2  -- is below 50000.
3  SELECT DEPARTMENT, MIN(SALARY) AS MINIMUM_SALARY FROM EMPLOYEES06
4  GROUP BY DEPARTMENT
5  HAVING MIN(SALARY) < 50000;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.003 seconds

	DEPARTMENT	MINIMUM_SALARY
1	HR	48000
2	Finance	47000

SQL Exercise 4 - Combined

Write a single query that shows:

- Department name
- Number of employees
- Average salary (rounded to 0 decimal places)
- Highest salary
- Lowest salary

Only show departments with more than 1 employee.

Order results by average salary descending.

[SQL Worksheet]*

Aa

1

-- # SQL Exercise 4 - Combined

2

3

SELECT DEPARTMENT, COUNT(*) AS TOTAL_COUNT,

4

ROUND(AVG(SALARY), 0) AS AVG_SALARY,

5

MAX(SALARY) AS MAXIMUM_SALARY,

6

MIN(SALARY) AS MINIMUM_SALARY FROM EMPLOYEES06

7

GROUP BY DEPARTMENT

8

HAVING COUNT(*) > 1

9

ORDER BY AVG_SALARY DESC;

10

11

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.001 seconds

	DEPARTMENT	TOTAL_COUNT	AVG_SALARY	MAXIMUM_SALARY	MINIMUM_SALARY
1	IT	4	75750	91000	55000
2	Finance	2	54000	61000	47000
3	HR	2	50000	52000	48000

LINUX PRACTICE

Linux Task 1 - cut

1. Extract only the name column (column 2).
2. Extract only the department column (column 3).
3. Extract name and city columns together (2 and 5).
4. Extract all columns except emp_id.

Hint: extract columns 2 to 5 using -f2-5.

5. Extract the first 3 characters of each line.

Use cut -c1-3.

Command: cut -d',' -f<column_number> filename

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f2 employees.csv
name
Rahul
Amit
Priya
Neha
Karan
Sneha
Rohan
Divya
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f3 employees.csv
department
IT
HR
IT
Finance
IT
HR
Finance
IT
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f2,5 employees.csv
name,city
Rahul,Pune
Amit,Mumbai
Priya,Pune
Neha,Delhi
Karan,Bangalore
Sneha,Mumbai
Rohan,Pune
Divya,Pune
```

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f2-5 employees.csv
name,department,salary,city
Rahul,IT,72000,Pune
Amit,HR,48000,Mumbai
Priya,IT,85000,Pune
Neha,Finance,61000,Delhi
Karan,IT,55000,Bangalore
Sneha,HR,52000,Mumbai
Rohan,Finance,47000,Pune
Divya,IT,91000,Pune
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -c1-3 employees.csv
emp
1,R
2,A
3,P
4,N
5,K
6,S
7,R
8,D
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```

Linux Task 2 - sort

1. Sort the file alphabetically by name.

Hint: sort by field 2.

2. Sort by salary in ascending order.

Hint: sort by field 4, use -t',' -k4.

3. Sort by salary in descending order.

Add the -r flag.

4. Sort by department name, then by salary

within each department.

Hint: use -k3 -k4.

5. Sort and save the output to sorted_employees.csv.

Command: sort -t',' -k<field> filename

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ sort -t',' -k2 employees.csv
2,Amit,HR,48000,Mumbai
8,Divya,IT,91000,Pune
5,Karan,IT,55000,Bangalore
4,Neha,Finance,61000,Delhi
3,Priya,IT,85000,Pune
1,Rahul,IT,72000,Pune
7,Rohan,Finance,47000,Pune
6,Sneha,HR,52000,Mumbai
emp_id,name,department,salary,city
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ sort -t',' -k4 employees.csv
7,Rohan,Finance,47000,Pune
2,Amit,HR,48000,Mumbai
6,Sneha,HR,52000,Mumbai
5,Karan,IT,55000,Bangalore
4,Neha,Finance,61000,Delhi
1,Rahul,IT,72000,Pune
3,Priya,IT,85000,Pune
8,Divya,IT,91000,Pune
emp_id,name,department,salary,city
```

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ sort -nr -k4 employees.csv
emp_id,name,department,salary,city
8,Divya,IT,91000,Pune
7,Rohan,Finance,47000,Pune
6,Sneha,HR,52000,Mumbai
5,Karan,IT,55000,Bangalore
4,Neha,Finance,61000,Delhi
3,Priya,IT,85000,Pune
2,Amit,HR,48000,Mumbai
1,Rahul,IT,72000,Pune
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ sort -t',' -k3 -k4 employees.csv
7,Rohan,Finance,47000,Pune
4,Neha,Finance,61000,Delhi
2,Amit,HR,48000,Mumbai
6,Sneha,HR,52000,Mumbai
5,Karan,IT,55000,Bangalore
1,Rahul,IT,72000,Pune
3,Priya,IT,85000,Pune
8,Divya,IT,91000,Pune
emp_id,name,department,salary,city
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```


Linux Task 3 - uniq

First extract the department column, then sort it, then apply uniq.

1. List all unique departments.
2. Count how many employees are in each department.

Hint: `uniq -c`

3. Find departments that appear more than once.

Hint: `uniq -d`

4. Extract the city column, sort it, and list unique cities with their count.

Note: `uniq` only works correctly on sorted input.

Always sort before using `uniq`.

Command: `cut | sort | uniq -c`

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f3 employees.csv | sort | uniq
Finance
HR
IT
department
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f3 employees.csv | sort | uniq -c
  2 Finance
  2 HR
  4 IT
  1 department
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f3 employees.csv | sort | uniq -d
Finance
HR
IT
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f5 employees.csv | sort | uniq -c
  1 Bangalore
  1 Delhi
  2 Mumbai
  4 Pune
  1 city
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ tail -n +2 employees.csv | cut -d',' -f5 | sort | uniq -c
  1 Bangalore
  1 Delhi
  2 Mumbai
  4 Pune
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$
```

Linux Task 4 - Pipeline Challenge

Using a single pipeline command for each:

1. List unique departments with employee count, sorted by count descending.

(cut → sort → uniq -c → sort -rn)

2. Find the number of employees from Pune.

(grep → wc -l)

3. Extract names of IT employees only.

(grep → cut)

4. List all unique cities alphabetically, one per line.

(cut → sort → uniq)

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f3 employees.csv | sort | uniq -c | sort -rn
4 IT
2 HR
2 Finance
1 department
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ grep -i "Pune" employees.csv | wc -l
4
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ grep -i "it" employees.csv | cut -d',' -f2
name
Rahul
Amit
Priya
Karan
Divya
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f5 employees.csv | sort | uniq
Bangalore
Delhi
Mumbai
Pune
city
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```